Maze Solving

BFS vs A* vs GBFS

Algorithm Comparison and Analysis

Elliot Lindsey and Nicolas Stamper

Problem Definition

What is Maze Solving?

Finding the shortest path from a start position to an end position in a graph with weighted edges.

Real-World Applications

Robotics
Autonomous movement and navigation.

Game Dev
Character AI and level layout.

Warehouses
Logistics and route planning.

Breadth-First Search (BFS)

How It Works

Starts from a source node and explores the graph level by level, visiting all neighbors at depth d.

In maze solving, uses a queue to iteratively explore neighbors, tracking visits to avoid cycles until goal is found or all paths are exhausted.

Strengths	Weaknesses
• Easy to implement • Always finds shortest path • Memory bounded by maze width	• Uses substantial memory at large scale • Explores many unnecessary nodes

A* Search

How It Works

Uses a heuristic function to estimate distance from current node to goal.

Prunes large portions of the search space, making it faster than BFS in practice.

Strengths	Weaknesses
• Finds the optimal path • Much faster than BFS in large environments • Widely used in industry (games, robotics)	• Requires careful heuristic design • Higher memory usage than GBFS

Greedy Best-First Search (GBFS)

How It Works

Expands the node that appears closest to the goal using only the heuristic, ignoring the actual cost so far.

Fast, but not guaranteed to find the shortest path, it can get trapped.

Models goal-directed behavior seen in nature, like animals navigating toward a visible target using only local information.

Strengths	Weaknesses
• Very fast with good heuristic • Simple to implement on A* framework • Low memory usage	• Not a guaranteed optimal path • Can fail or get trapped in corridors

Experimental Design

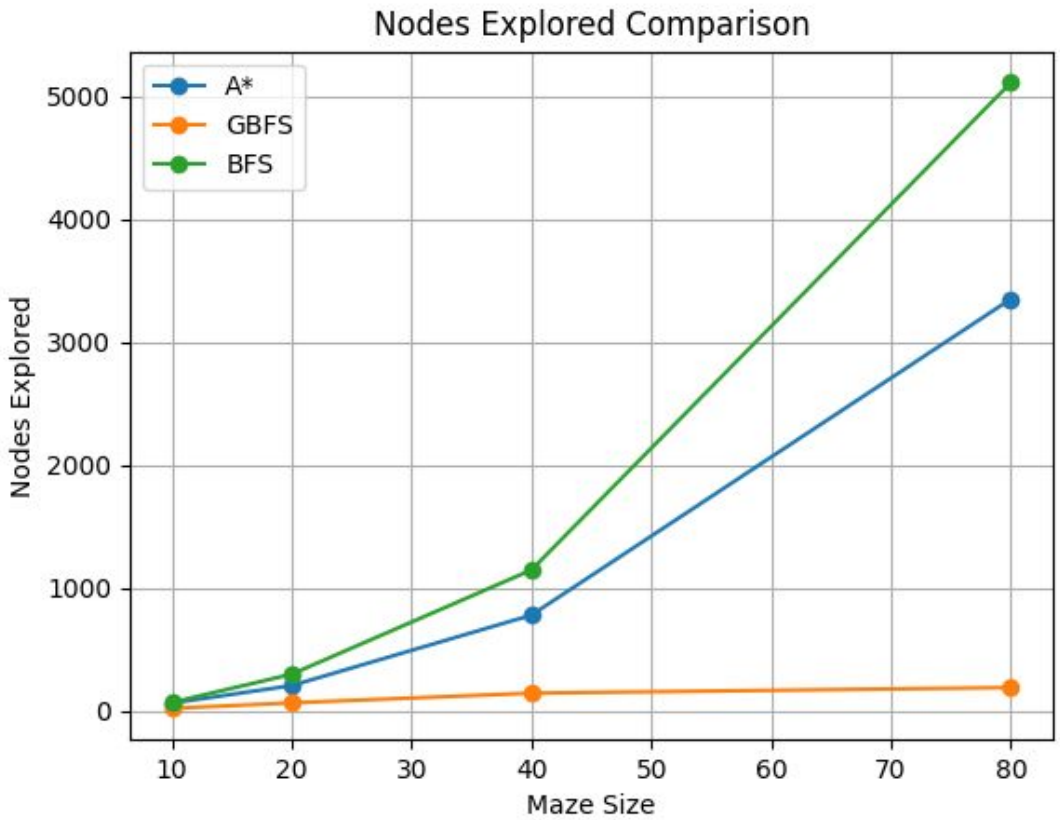
Setup and Parameters

Parameter	Value
Maze Sizes (n x n)	10, 20, 40, 80
Trials	20
Random Seed	Fixed (for reproducibility)
Maze type	Randomized grid-based
Valid path guarantee?	Yes (generated algorithmically)

Metrics Measured

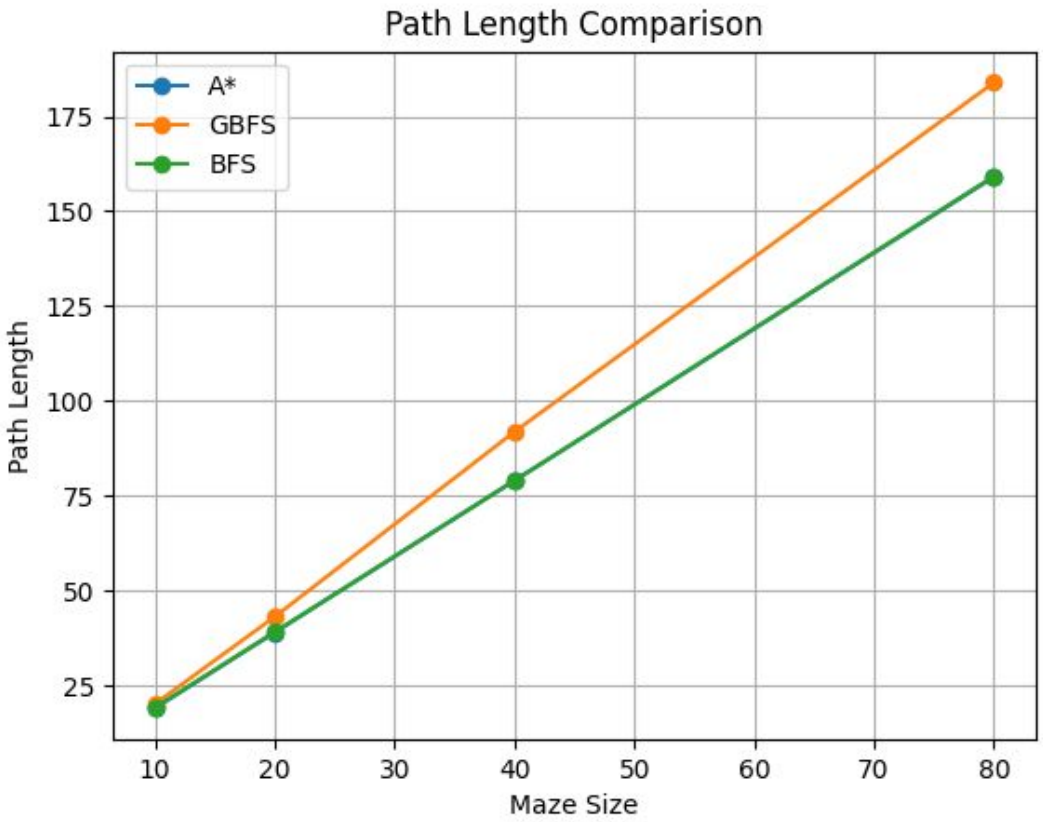
Runtime Execution time per trial
Nodes Explored Average nodes visited
Path Length Solution quality
Success Rate % of mazes solved

Nodes Explored



BFS **5,100** 80x80
A* **3,350** 80x80
GBFS **210** 80x80

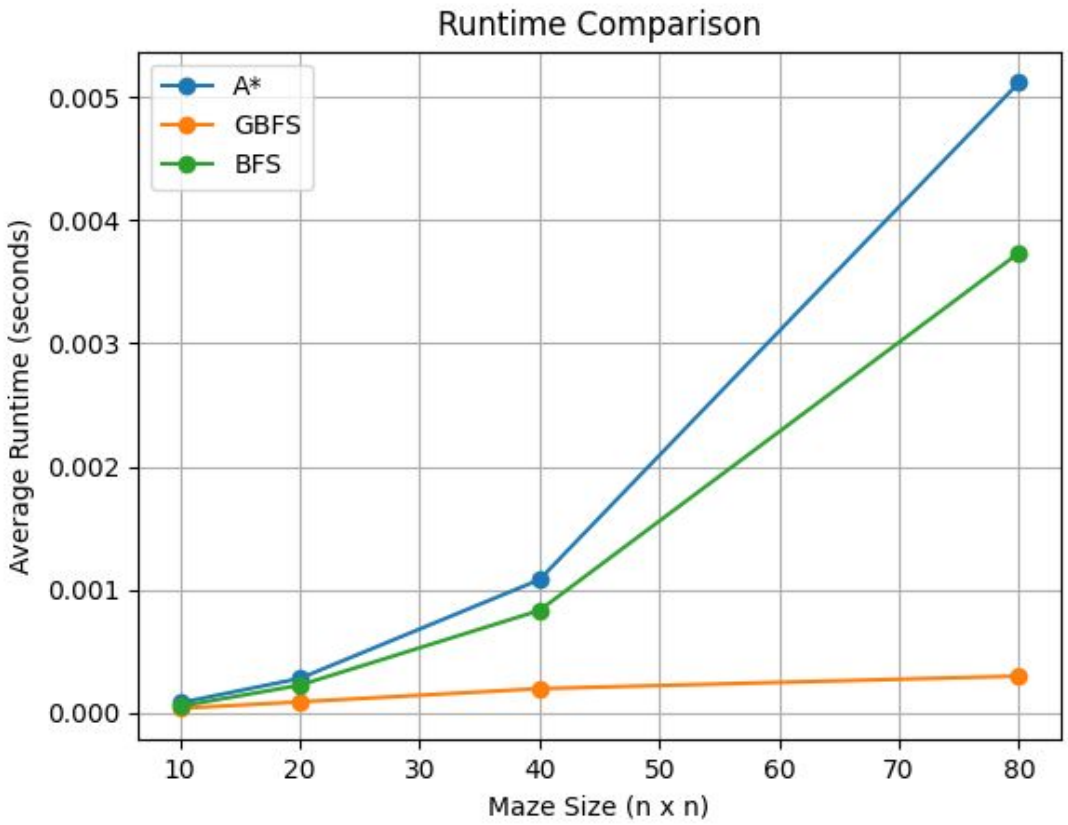
Path Length



A* and BFS produce identical path lengths.

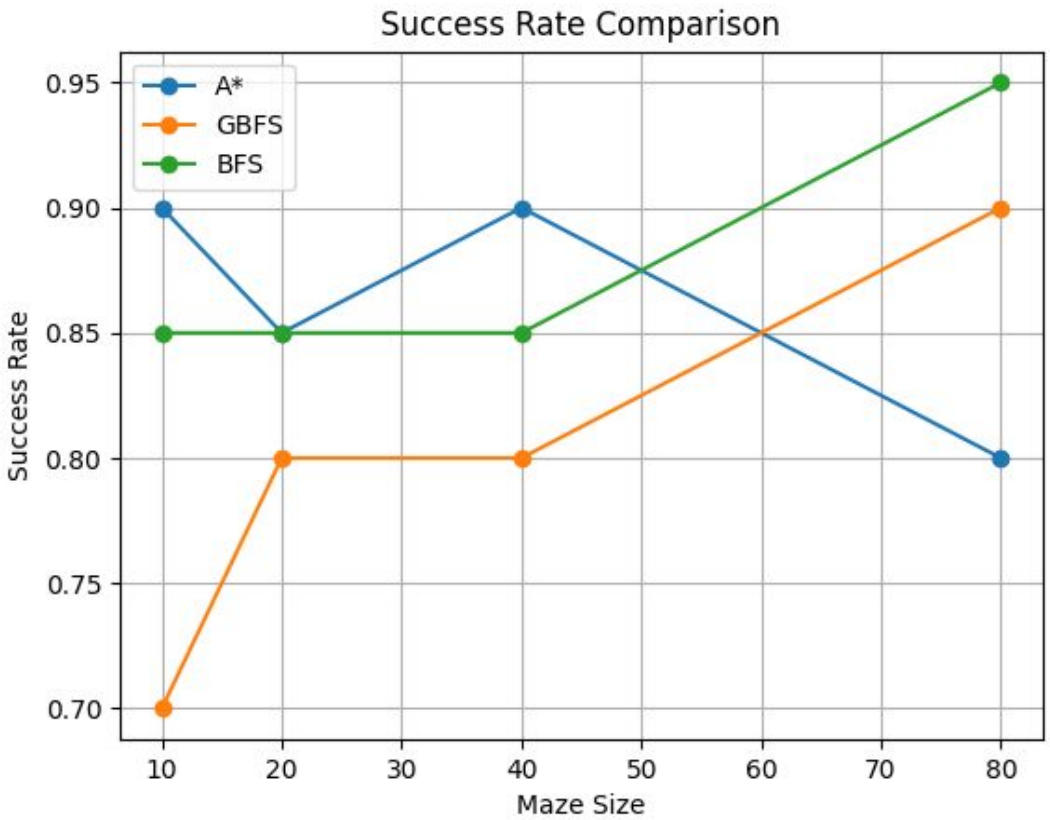
GBFS paths are longer due to its greedy nature.

Runtime



A*   **0.0053s**   Highest runtime at 80x80
BFS   **0.0038s**   Mid-range runtime at 80x80
GBFS   **0.0003s**   Much faster runtime at 80x80

Success Rate



BFS success rate improves with larger mazes.

A* starts strong, but dips at 80x80.

GBFS starts the lowest, but catches up.

Comparisons

Metric	BFS	A*	GBFS
Nodes Explored	5,100	3,350	210
Path Length	Optimal	Optimal	Sub-optimal
80x80 Runtime	0.0038s	0.0053s	0.0003s
Success Rate	85–95%	80–90%	70–90%
Guarantees Optimal?	Yes	Yes	No
Memory Usage	High	Medium	Low

Challenges and Design Decisions

ACO → GBFS Ant Colony Optimization was planned but replaced with GBFS due to high compute costs.	**GBFS Duplicate Node** GBFS suffered from duplicate nodes, inflating runtime. Fixed by managing visited sets and preventing repeated insertions.
Reproducibility Maze generator uses Python's random module. A fixed random seed was used to ensure reproducibility.	**Architecture** All algorithms are interchangeable with a shared experiment runner.

Conclusion

GBFS: Speed and Memory Explores fewer nodes than BFS and runs faster at 80x80.	**BFS and A*: Optimality** Both produce identical shortest paths.
BFS: Reliability at Scale Success rate is 95% at 80x80, higher than A* or GBFS.	**Trade-Offs** No single algorithm dominates all metrics.